

Interfaces Gráficas de Usuario en Java

Programación Orientada a Objetos

José Francisco Vélez Serrano



- Breve historia de los GUI
- Los kit de desarrollo de GUI en Java
- Componentes Swing
- Eventos y listeners
- Separación de la GUI y la lógica

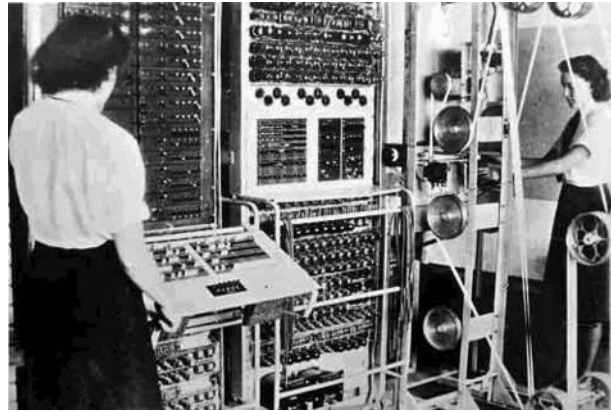
Breve historia de los GUI

En esta sección se realiza una introducción histórica a las interfaces gráficas de usuario (GUI).

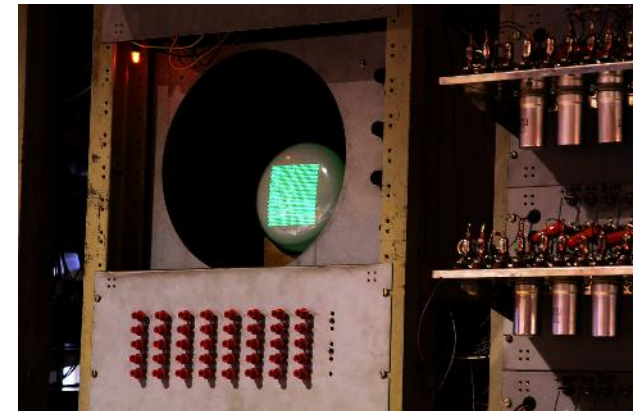
Breve historia de los GUI

Investigación en GUI

1943 - Colossus Mark 1
Primer ordenador electrónico digital
programable mediante cableado



1948 - SSEM (Baby)
Primer ordenador electrónico digital programable



Breve historia de los GUI

Investigación en GUI

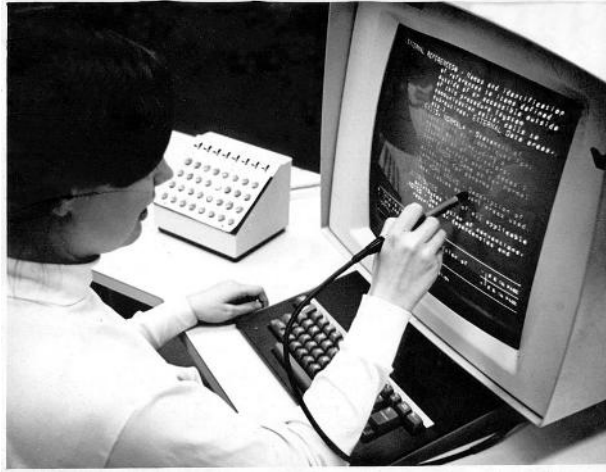


Imagen de 1969

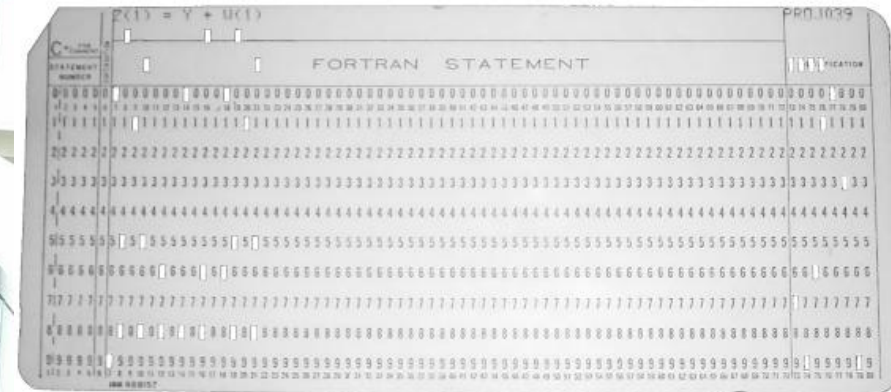
Colossus Mark 1 - 1943

SSEM (Baby) - 1948

1952 – Lápiz Óptico, MIT

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

1952 – El IBM 701, primer ordenador comercial, usa tarjetas perforadas
Las tarjetas perforadas existían desde 1804 para los telares

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

1967 - Douglas Engelbart, Stanford Research Institute

- Wheel mouse
- oN-Line System (NLS)
 - Hypertext
 - Windows

Breve historia de los GUI

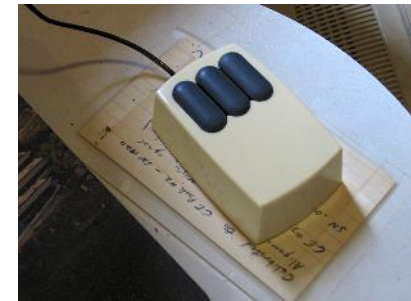
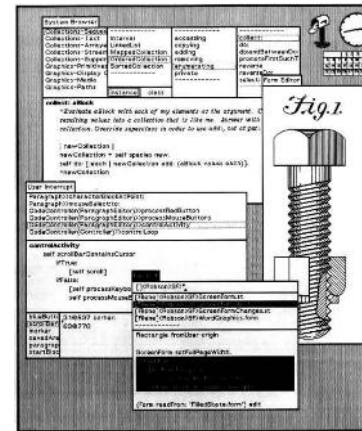
Investigación en GUI



- Colossus Mark 1 - 1943
- SSEM (Baby) - 1948
- Tarjetas perforadas - 1952
- Lápiz Óptico - 1952
- Douglas Engelbart - 1967
- 1968 – Sutherland y Bob Sproull crean las primeras gafas de realidad virtual

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

Douglas Engelbart - 1967

Sutherland y Sproull - 1968

1972 - Xerox Parc, Palo Alto Research Centre
Metáfora de escritorio – Alan Kay – 1970
Ball mouse (Bill English)
Alto personal computer

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

Douglas Engelbart - 1967

Sutherland y Sproull - 1968

Xerox Alto - 1972

1983 - Apple
• Lisa (1983)

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

Douglas Engelbart - 1967

Sutherland y Sproull - 1968

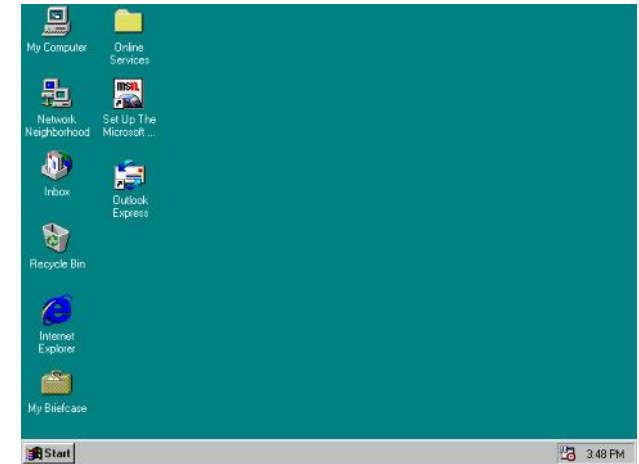
Xerox Alto - 1972

Apple - 1983

1984 – Macintosh
Unix - X

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

Douglas Engelbart - 1967

Sutherland y Sproull - 1968

Xerox Alto - 1972

Apple - 1983

X - 1984

1985 - Microsoft

- Windows 1.0 (1985)
- Windows 3.1 (1992)
- Windows 95

Breve historia de los GUI

Investigación en GUI



Colossus Mark 1 - 1943

SSEM (Baby) - 1948

Tarjetas perforadas - 1952

Lápiz Óptico - 1952

Douglas Engelbart - 1967

Sutherland y Sproull - 1968

Xerox Alto - 1972

Apple - 1983

X - 1984

Windows 1.0 - 1985

IBM Angler - 1992

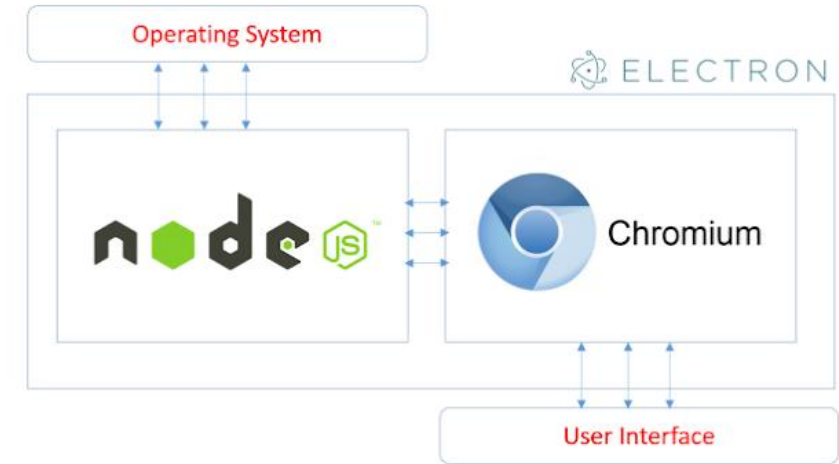
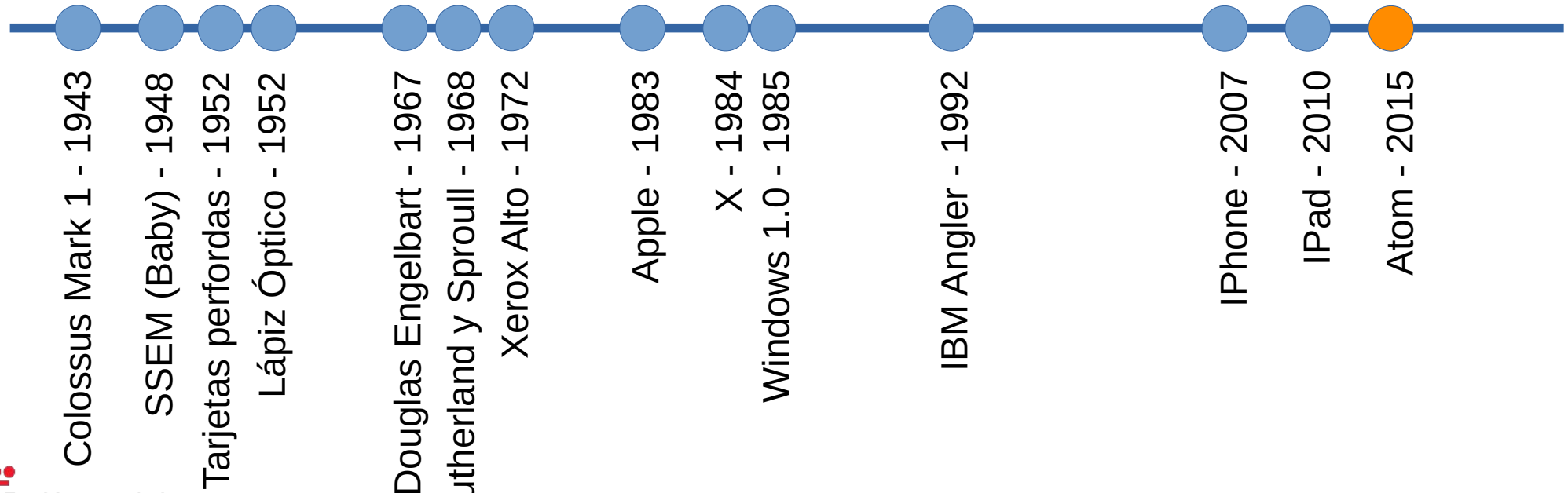
Breve historia de los GUI

Investigación en GUI



Breve historia de los GUI

Investigación en GUI



Breve historia de los GUI

Soluciones comerciales

La interfaz de usuario está en continua evolución:

- 40-70 - Cableado e interruptores
- 70-80 - Impresora y las fichas perforadas
- 80-90 - Teclados y las pantallas bicolor
- 90-00 - Pantallas a color, teclado y ratón
- 00-10 - Pantallas táctiles
- 10-20 - Voz Gestos

Breve historia de los GUI

Justificación de las GUI

La interfaz de texto no contempla el uso de punteros de pantalla y presenta la información en una única pantalla de manera secuencial utilizando únicamente elementos de tipo texto alfanumérico.

Las **Interfaces Gráficas de Usuario** (GUI) contemplan el uso del ratón, permiten presentar la información en múltiples pantallas virtuales (llamadas ventanas) y además permiten mayores posibilidades de representación de elementos gráficos distintos a los caracteres alfanuméricos.

Los kit de desarrollo de GUI en Java

En esta sección se introducen los diferentes kits para el desarrollo de Interfaces Gráficas de Usuario (GUI) que han existido en Java.

Los kit de desarrollo de GUI en Java

AWT

Hasta la versión 1.2 de Java el paquete de gestión de interfaces gráficos era **AWT** (Abstract Window Toolkit).

 Label

 Button

 Text Field

 Text Area

 Checkbox

 Choice

 List

 Scrollbar

 Scroll Pane

 Panel

 Canvas

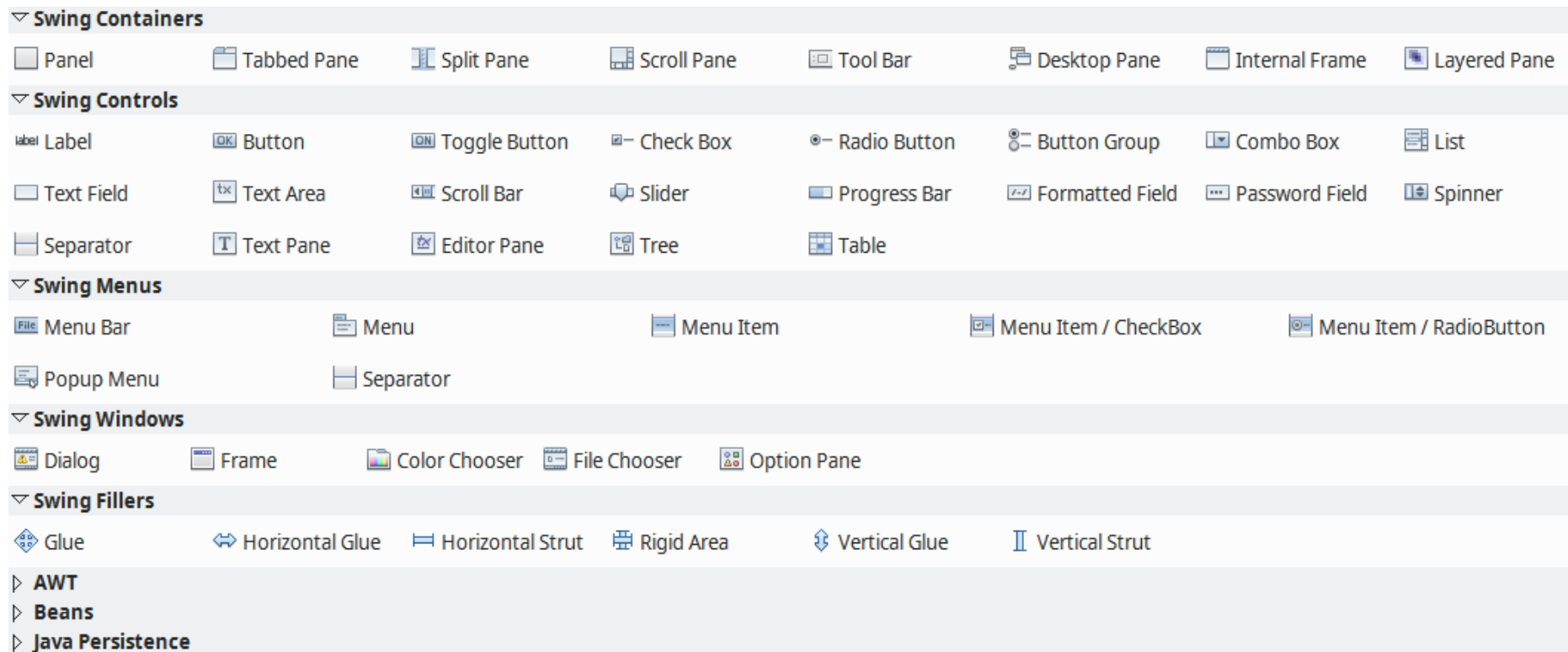
 Menu Bar

 Popup Menu

Los kit de desarrollo de GUI en Java

Swing

En la versión 1.2 apareció **Swing**, que es mucho más avanzado. Aunque Swing se ejecuta sobre AWT, Swing lo sustituye excepto en los applets (en desuso).

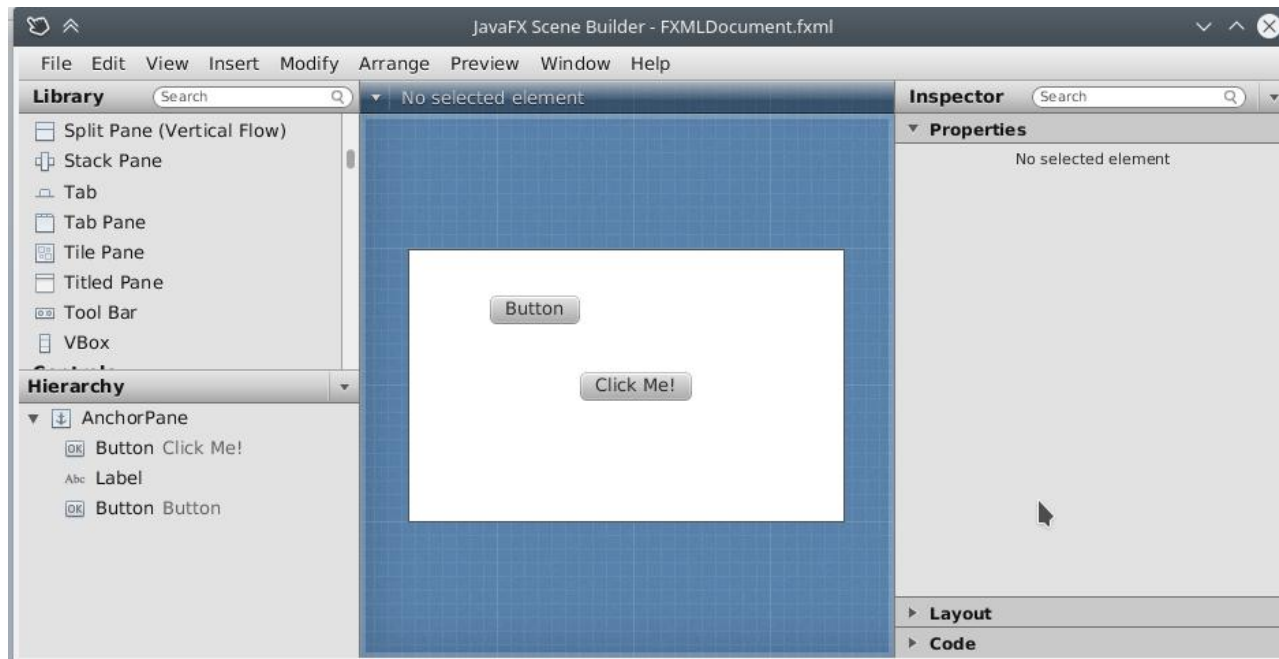


Los kit de desarrollo de GUI en Java

JavaFX

En el 2012 Oracle decidió añadir **JavaFX** que añade elementos como 3D o personalización por CSS.

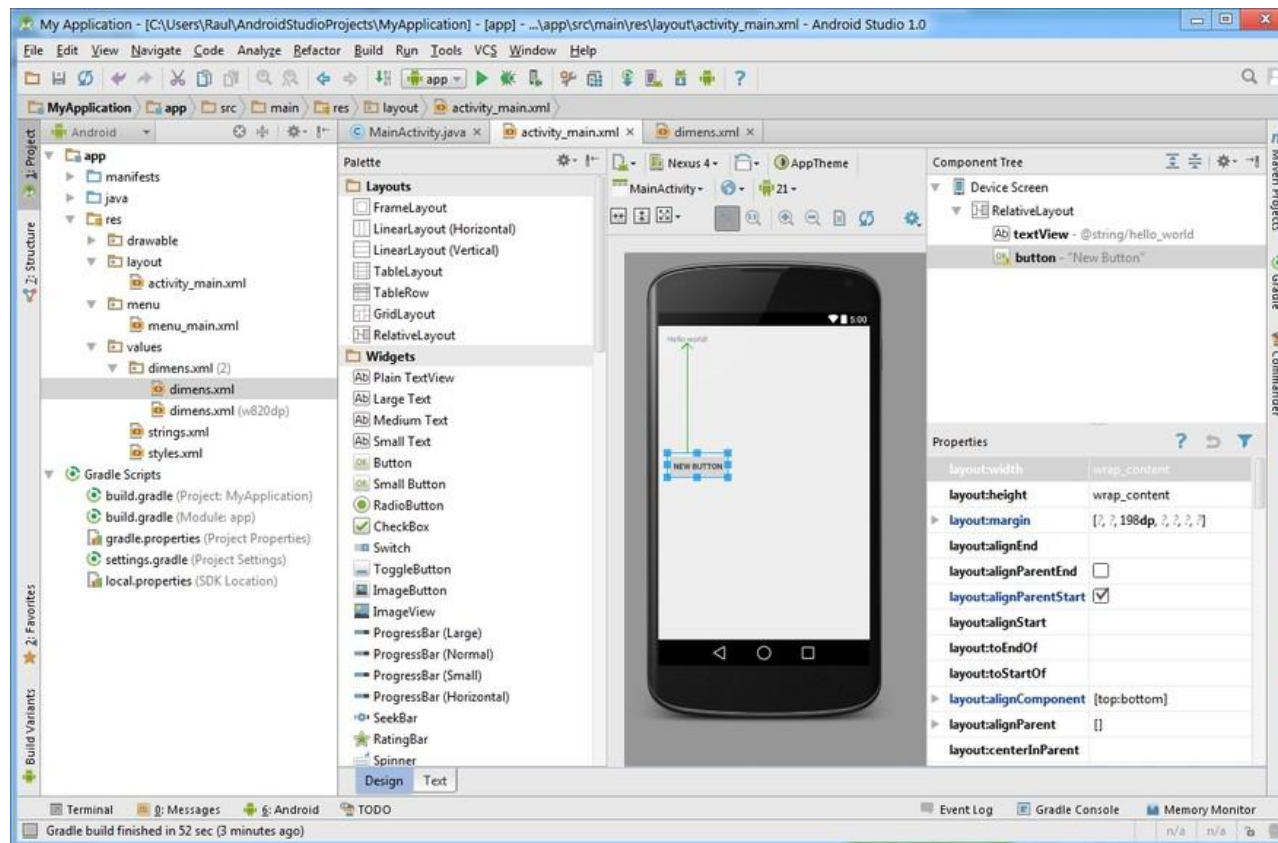
Desde el JDK 11 JavaFX ya no forma parte del core y debe instalarse por separado.



Los kit de desarrollo de GUI en Java

Android

Hasta hace poco, el lenguaje de referencia en Android también era un Java creado por Google junto al IDE Android Studio.



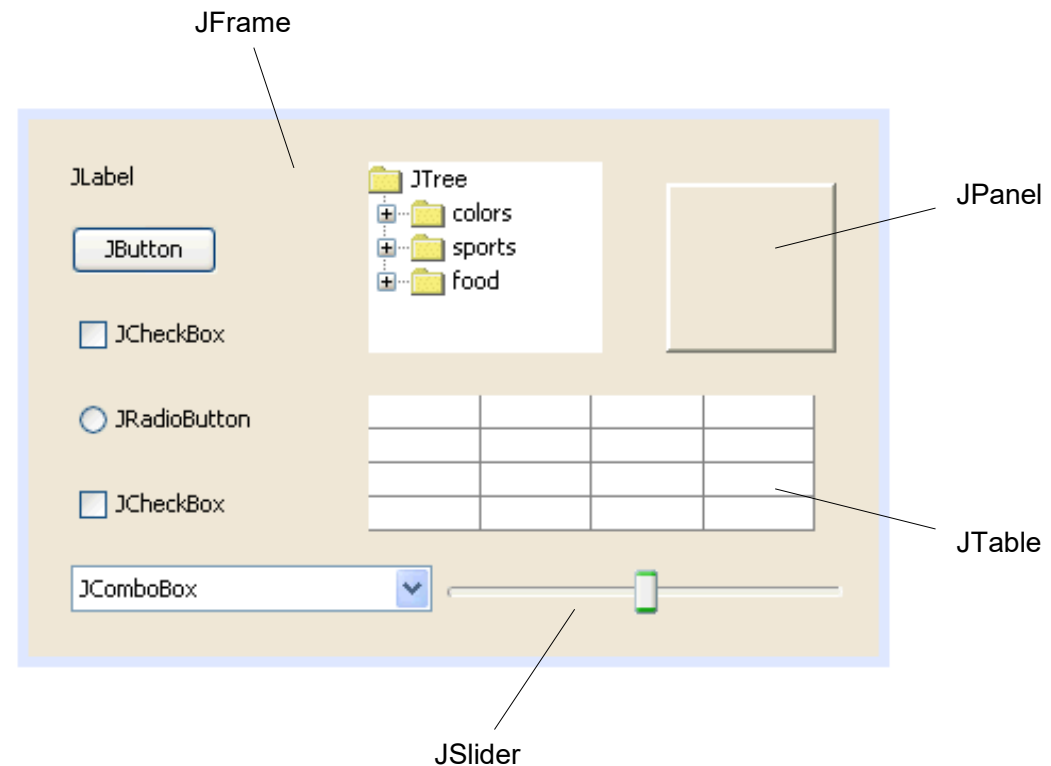
Estructura de los componentes Swing

En esta sección se presenta la base del kit de desarrollo de GUI Swing

Estructura de los componentes Swing

Component

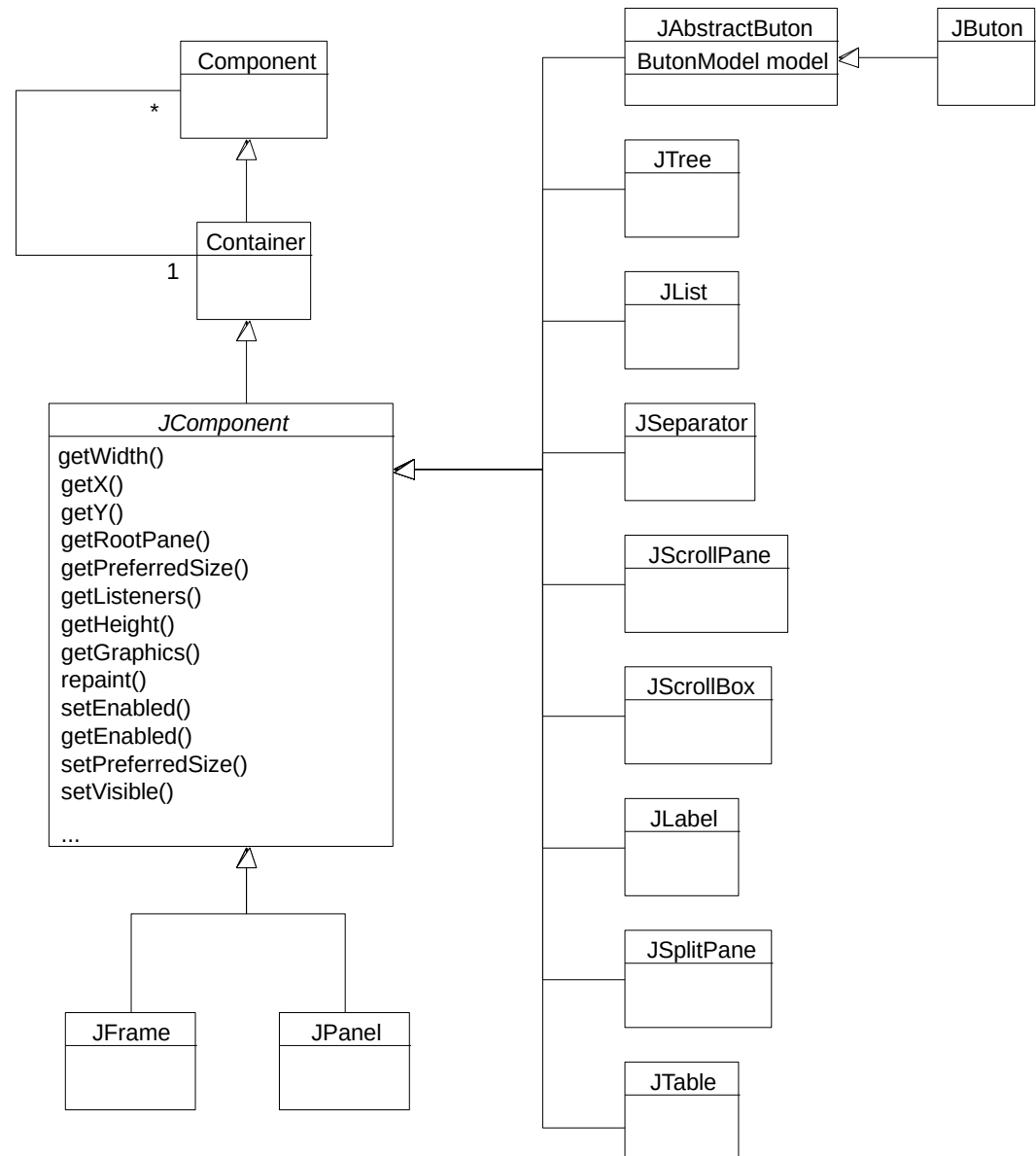
- La interfaz gráfica de Java está constituida de objetos **Component**.
- Estos componentes son los instrumentos que el usuario podrá utilizar para comunicarse con un programa.



Estructura de los componentes Swing

Componentes habituales

- Los componentes más habituales son los botones, los campos de escritura de texto, las etiquetas, las listas, las cajas de selección y las barras de desplazamiento.
- Swing permite crear componentes mediante **herencia**.



Estructura de los componentes Swing

Uso de Netbeans para diseñar GUI

Para crear JFrames

Árbol de componentes en el diseño

Paleta de componentes

Zona de visualización e interacción con el diseño

Propiedades del componente seleccionado

Calculator - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Search (Ctrl+I)

Projects... Files Services

Calculator

Source Packages

calculator

NewJFrame.java

Source Design History

New

Find... Ctrl+F

Cut Ctrl+X

Copy Ctrl+C

Paste Ctrl+V

Delete Delete

Refactor

Compile Package F9

Test Package Ctrl+F6

Run Selenium Tests

History

Tools

Folder...

JFrame Form...

Java Class...

Java Package...

Java Interface...

JPanel Form...

Entity Class...

Entity Classes from Database...

Other...

Swing Containers

Panel

Split Pane

ToolBar

Internal Frame

Tabbed Pane

Scroll Pane

Desktop Pane

Layered Pane

Swing Controls

Label

Button

jButton2 [JButton] - Properties

Properties

Binding

Events

Code

Properties

action <none>

background [239,240,241]

font Noto Sans 13 Plain

foreground [35,38,39]

icon <none>

mnemonic

jButton2 [JButton]

Output Test Results Javadoc Terminal - jvelez@rota: ~

Estructura de los componentes Swing

Contenedores

En Swing los objetos Component siempre deben localizarse dentro de objetos de la clase **Container**.

Hay dos clases tipos de contenedores:

- Los **JFrame** son ventanas, con todos los elementos característicos de estas: bordes, títulos, gadgets para agrandarlas y encogerlas, etc.
- Los **JPanel** simplemente define un espacio donde disponer otros componentes.

Cada contenedor tiene un **LayoutManager** (manejador de composición) que se encarga de disponer los objetos Component que contiene

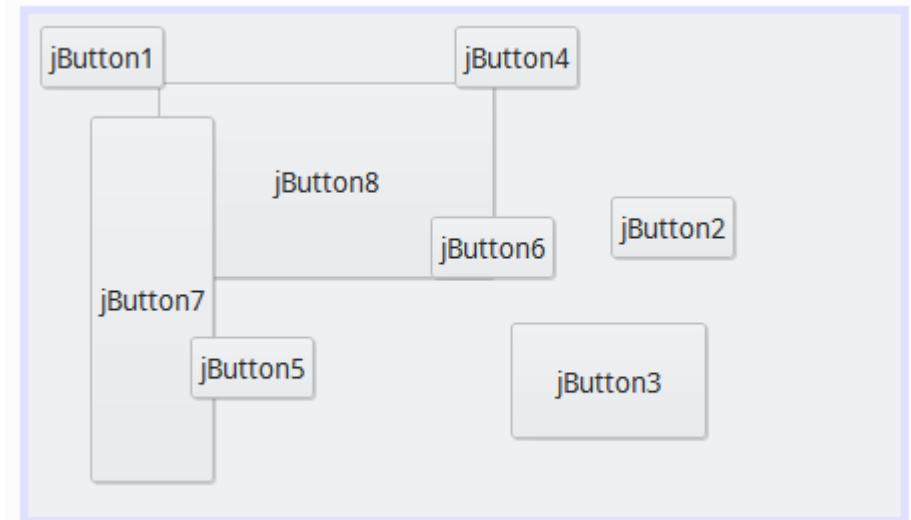
Estructura de los componentes Swing

AbsoluteLayout

AbsoluteLayout coloca los componentes en posiciones fijas respecto a la esquina superior izquierda del contenedor.

Este Layout es el más sencillo, pero no modifica los componentes al cambiar el tamaño del contenedor.

Se suele utilizar para paneles o ventanas que no admitan cambios de tamaño.



Estructura de los componentes Swing

FlowLayout

FlowLayout los componentes fluyen dentro del contenedor en que estén de izquierda a derecha y si no caben de arriba a abajo.

Este Layout hace que si se modifica el tamaño del contenedor, los componentes fluyan, pero no cambian de tamaño.

Este layout se suele utilizar para paneles que contengan barras de botones.



Estructura de los componentes Swing

BorderLayout

BorderLayout sitúa los componentes en cada uno de los puntos cardinales de un Contenedor o en el centro (expandiéndose para ocupar todo el área posible). Como máximo muestra 5 componentes.

Si se modifica el tamaño del contenedor, este layout reescala los componentes.

Este layout se suele usar para situar los paneles principales internos de una JFrame.



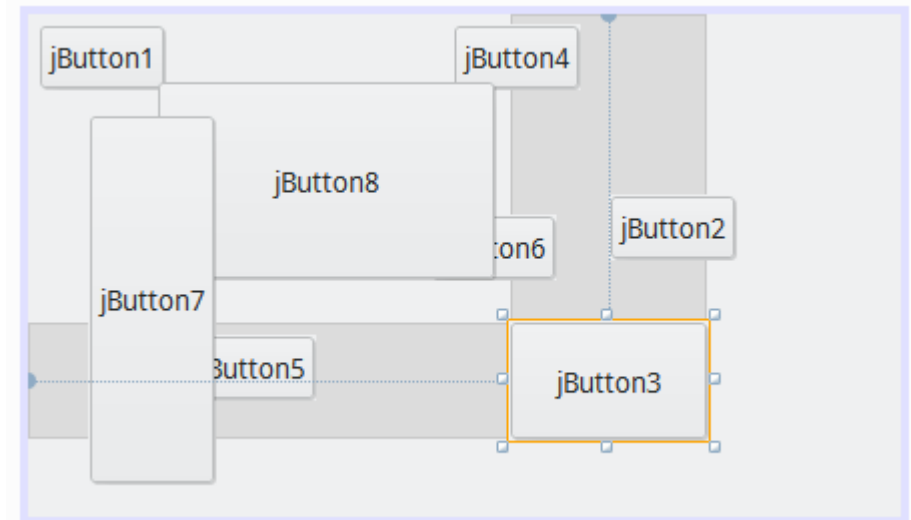
Estructura de los componentes Swing

FreehandLayout

Java tiene otros gestores de Layout más versátiles (como **GridLayout** y **GridBagLayout**), aunque su uso resulta complejo.

El diseñador gráfico de Netbeans 5.0 junto al Layout **Freehand** simplificó enormemente el diseño introduciendo anclas.

Las anclas fijan distancias que no pueden cambiar cuando los elementos escalables cambian.

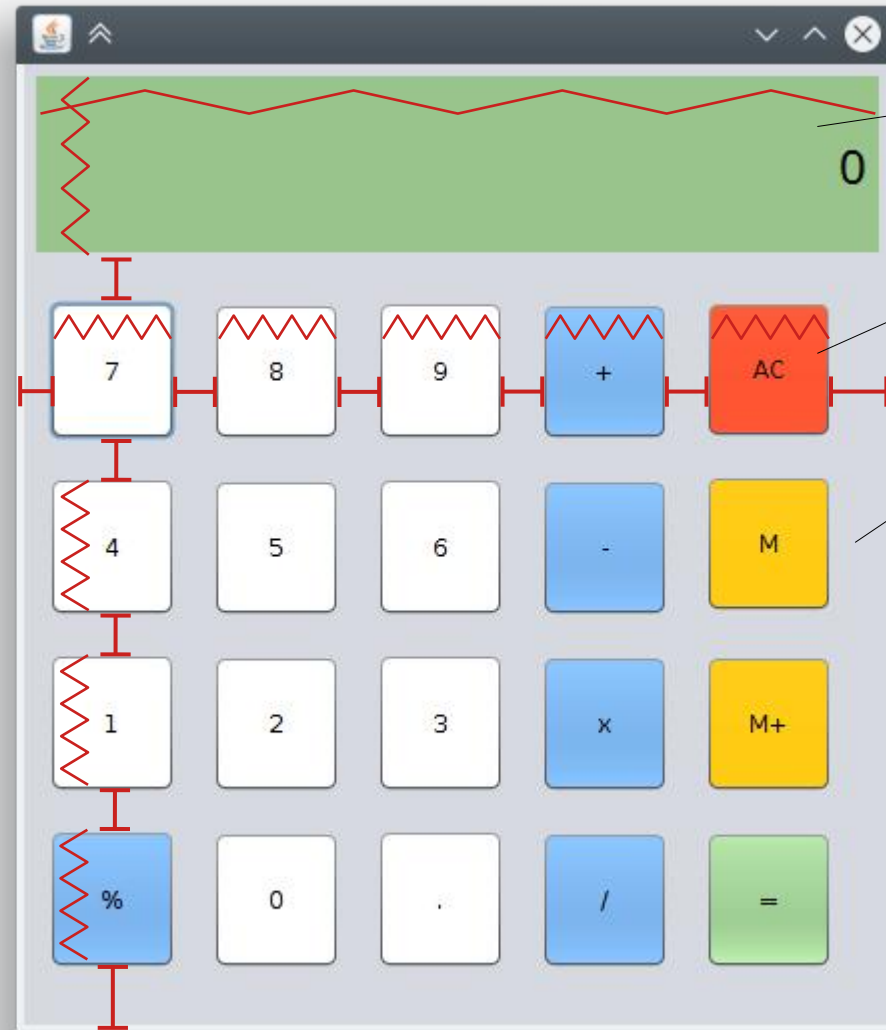


Estructura de los componentes Swing

Ejemplo de calculadora con FreeLayout


Elemento escalable


Ancla



JLabel

JButton

JFrame con
FreehandLayout

Eventos y listeners

En esta sección se presentan los eventos como la forma en la que Swing se comunica con el programa y los Listener como el mecanismo de detección de eventos.

Eventos y listeners

Los eventos

La gestión de la GUI de la mayoría de los sistemas actuales utilizan el **paradigma de eventos**.

Un evento es el resultado de una interacción del usuario con la interfaz gráfica.

AWT define un conjunto de eventos, cada uno de los cuales describe con cierto grado de precisión el resultado de las interacciones de un usuario.

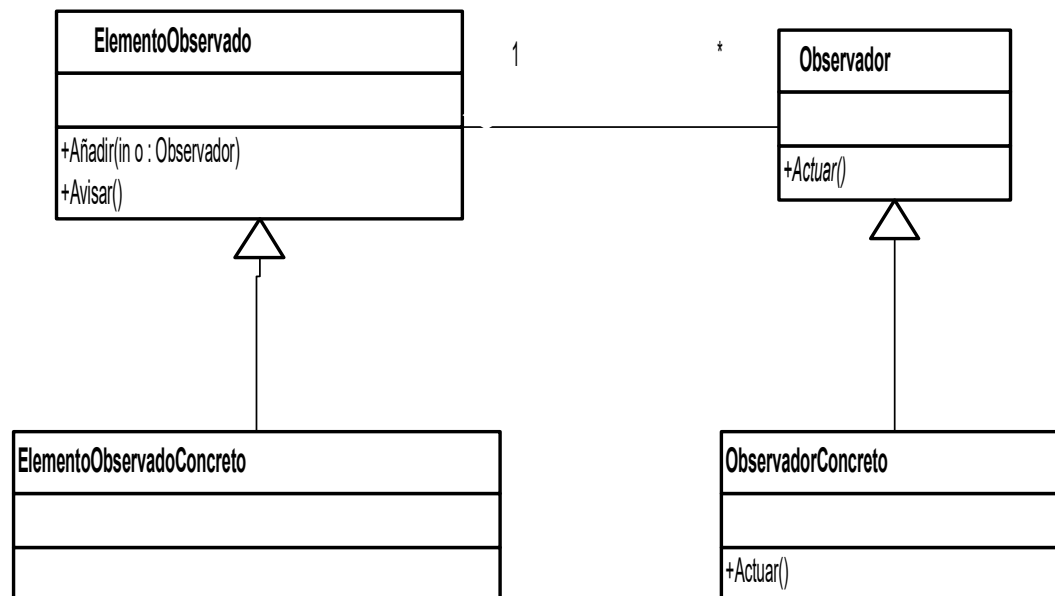
Los programas con interfaz gráfica se encargan de recibir estos eventos cuando se producen y de generar las respuestas adecuadas.

Eventos y listeners

Patrón observador

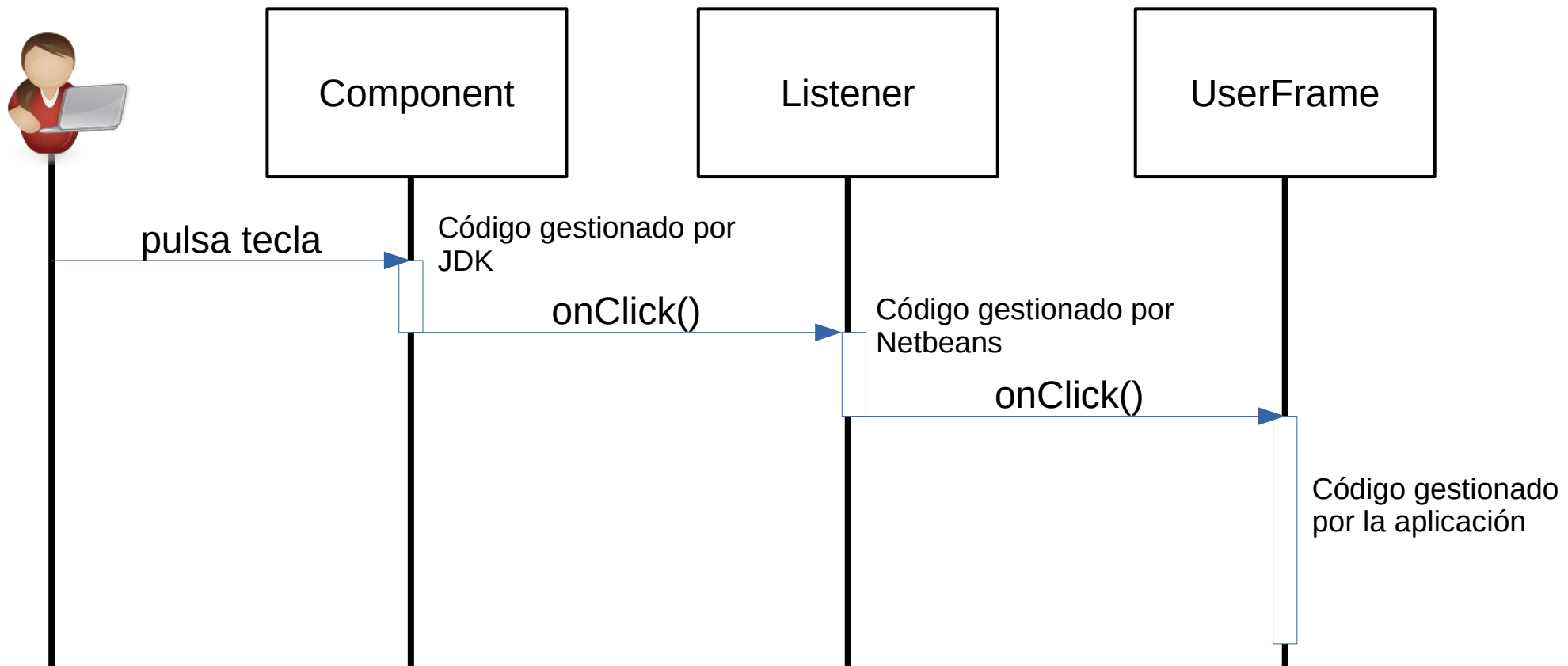
Un listener es un objeto que sigue el **patrón observador**.

Este patrón crea una relación entre objetos en la que uno de ellos es observado por el resto, de manera que cuando el objeto observado cambia el resto puede automáticamente realiza alguna acción



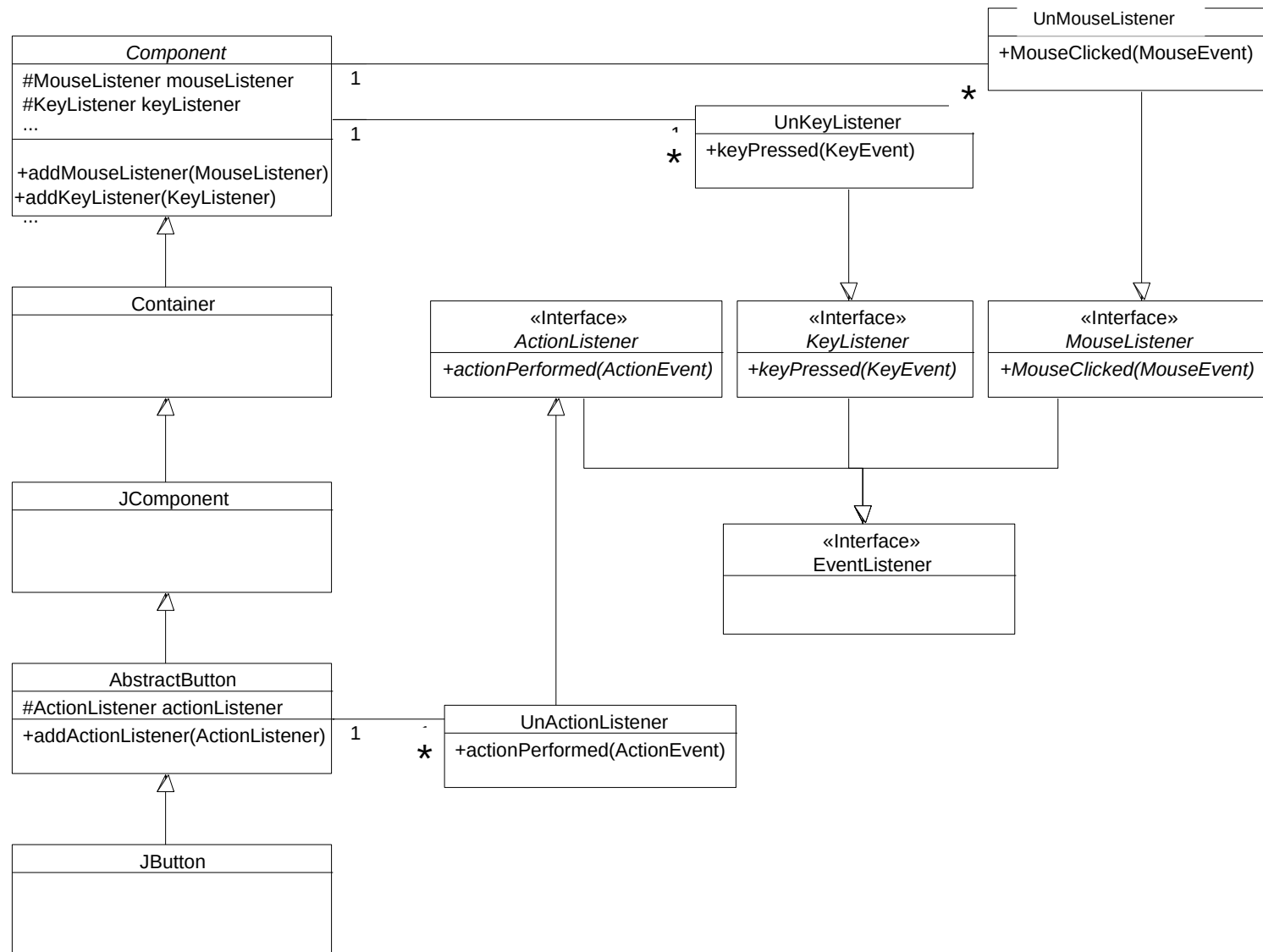
Eventos y listeners

Secuencia de eventos



Eventos y listeners

Uso de los listener



Eventos y listeners

El thread de gestión de eventos

El tratamiento de los eventos en Java se realiza desde un thread especial conocido como el Event Dispatch Thread.

Para saber si estás en el Event Dispatch Thread puedes invocar a `isEventDispatchThread` en `javax.swing.SwingUtilities`.

Los listener se ejecutan en este thread especial.

Es importante que todo lo que se ejecute en este thread sea breve, para evitar dejar colapsado el GUI.

Separación de la GUI y la lógica

En esta sección se explica la necesidad de separar la GUI de una aplicación de su lógica y se presenta una forma de hacerlo basado en el patrón Modelo-Vista-Controlador.

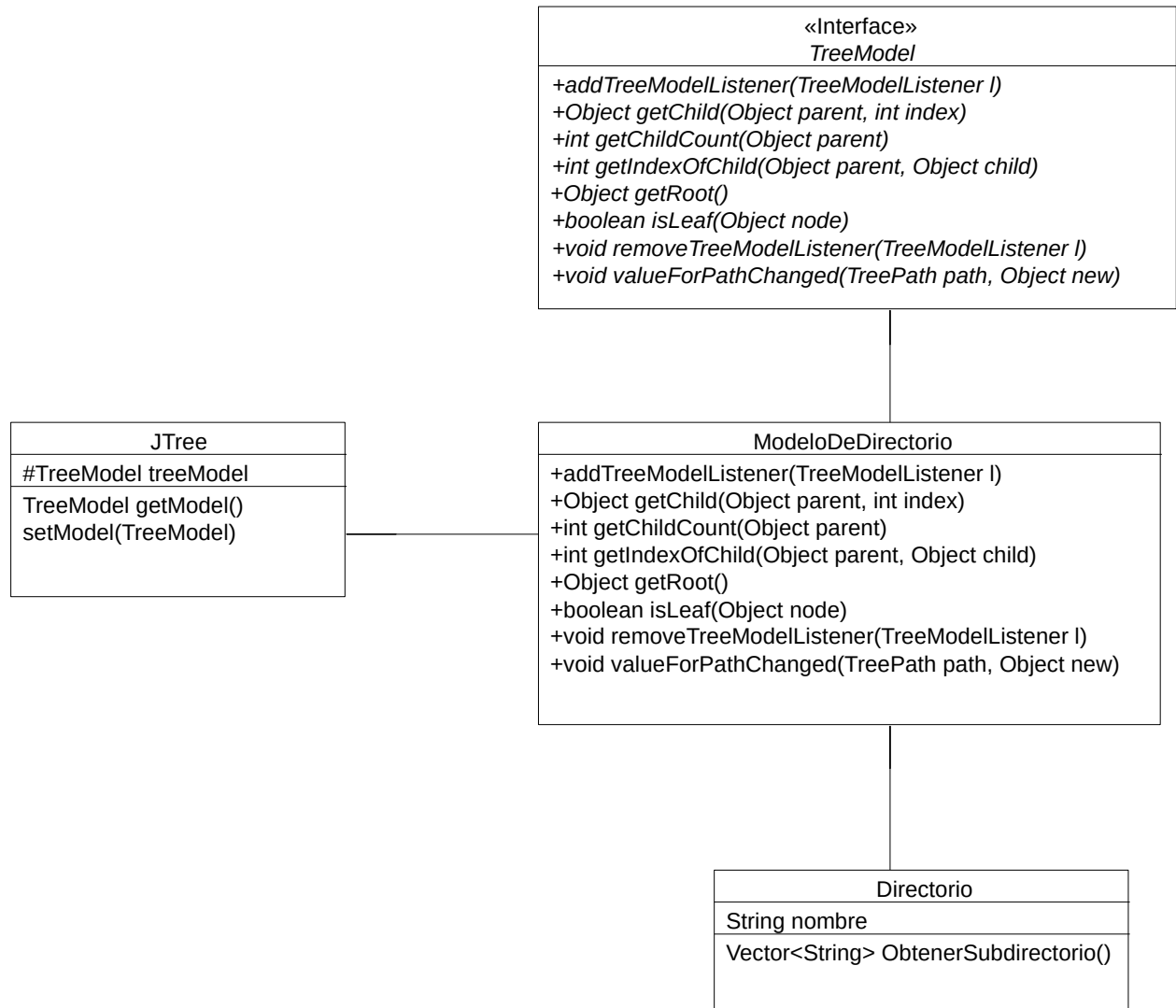
Separación de la GUI y la lógica

Modelo Vista Controlador

Los componentes de Swing permiten representar la información que contienen mediante una arquitectura que separa el componente gráfico (la vista) del modelo subyacente a la información mediante un controlador.

En componentes como JList, JTree y JTable, esta característica es especialmente útil, ya que facilita de manera sorprendente su uso.

El componente representa la información mediante la interrogación de una clase que controla al modelo que realmente contiene los datos.



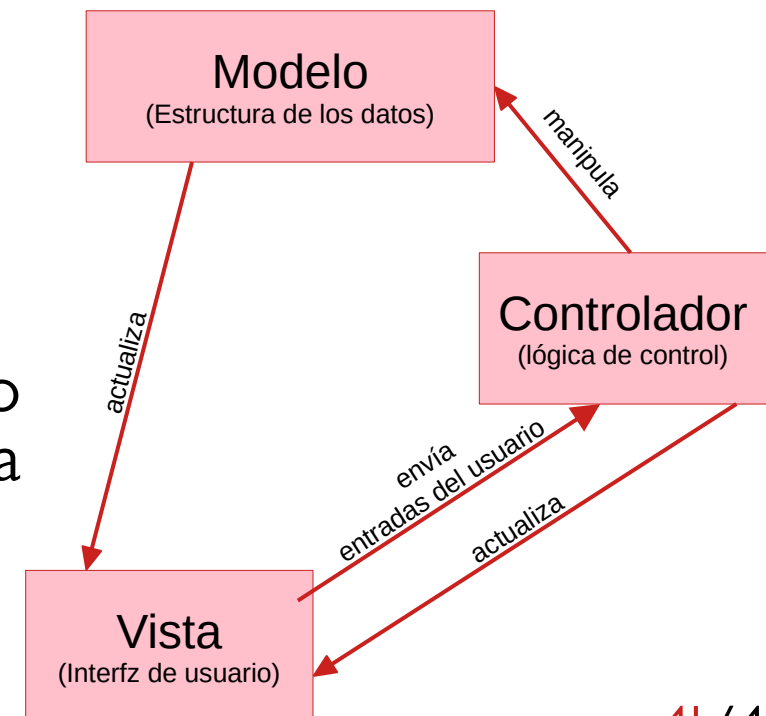
Separación de la GUI y la lógica

Patrón Modelo Vista Controlador

El patrón Modelo-Vista-Controlador (MVC) es un Patrón de diseño utilizado a menudo para desarrollar interfaces de usuario.

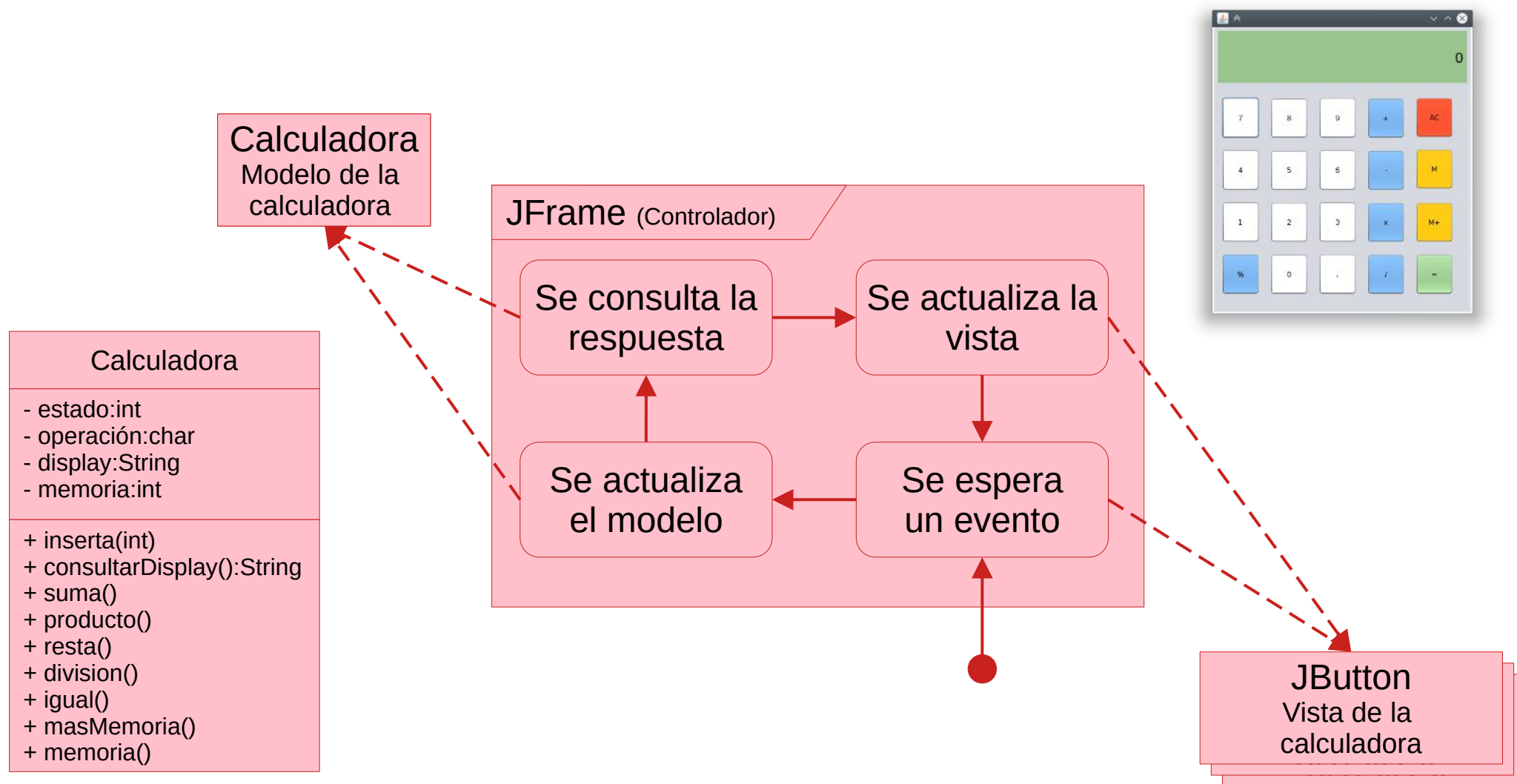
Se utiliza para separar la lógica de la aplicación y la representación interna de la información de la forma en que esa información se presenta al usuario o la forma en que se captura.

- El **modelo** es el componente central. Maneja los datos, la lógica y las reglas de la aplicación.
- La **vista** es cualquier representación de la información. Una tabla, un listado...
- El **controlador** acepta entradas del usuario y las transforma en comandos para la vista o el modelo.



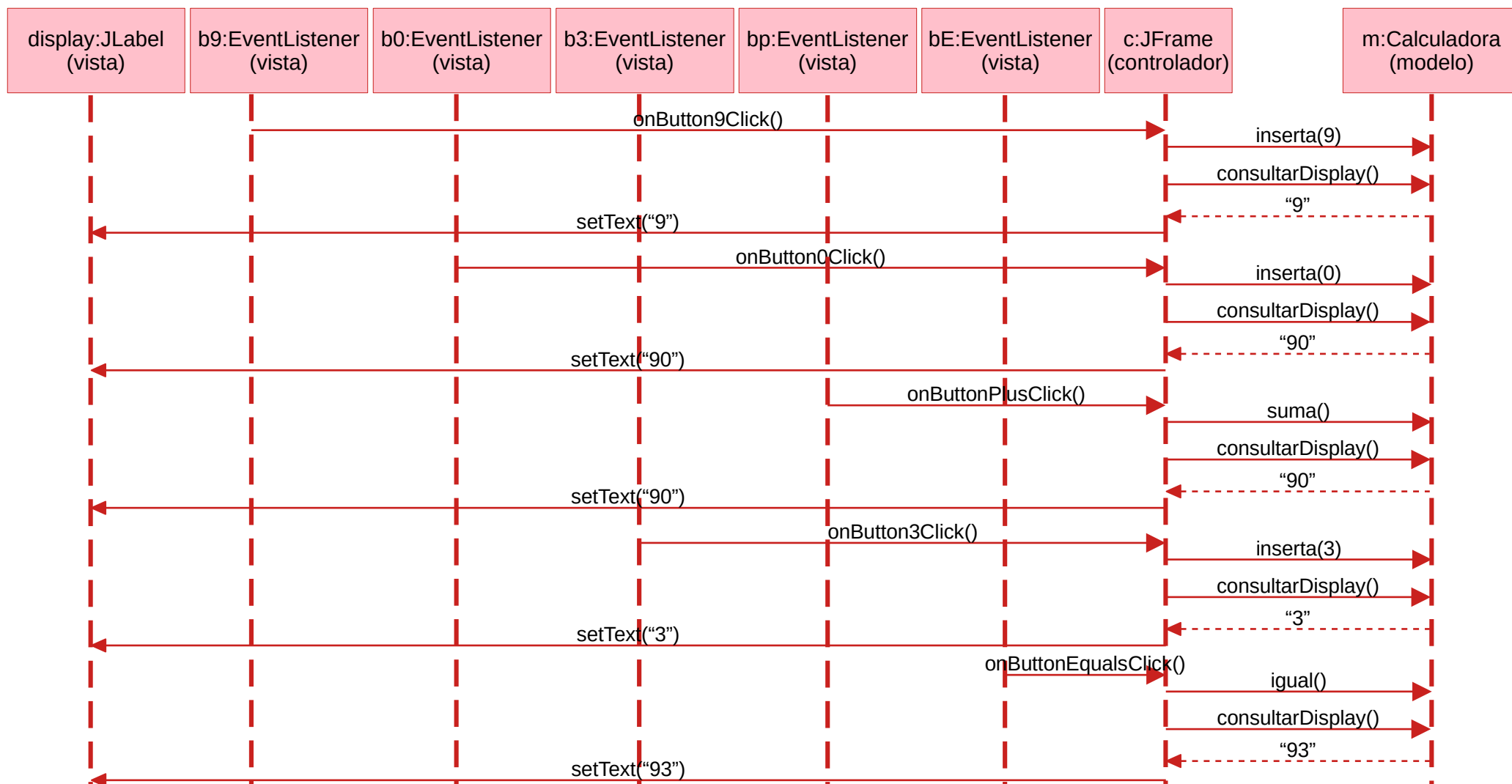
Separación de la GUI y la lógica

Ejemplo de la calculadora



Separación de la GUI y la lógica

Ejemplo de la calculadora



Referencias

- Jenifer Tidwell y otros, Designing Interfaces: Patterns for Effective Interaction Design, 2020.
- Steve Krug, Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability, 2013.
- Everett N. McKay, UI is Communication: How to Design Intuitive, User Centered Interfaces by Focusing on Effective Communication, MK, 2013.
- Wilbert O. Galitz, The Essential Guide to User Interface Design: An Introduction to GUI Design Principles and Techniques, 2007.
- Eric Butow, User Interface Design for Mere Mortals, 2007.

Licencia de publicación en acceso abierto



Este documento está publicado bajo la licencia Creative Commons Atribución 4.0 Internacional (CC BY 4.0) <https://creativecommons.org/licenses/by/4.0/>

Los autores han dado su consentimiento para esta publicación en acceso abierto. Se permite copiar, distribuir, remezclar, transformar y construir a partir del material, incluso con fines comerciales, siempre que se dé el crédito correspondiente.